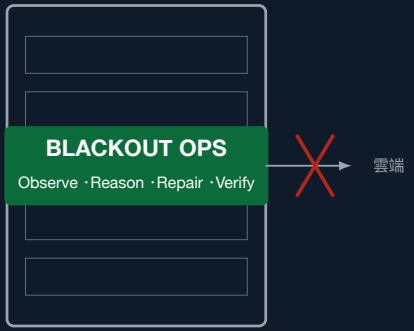
Blackout Ops

當基礎設施出問題時，Agent 仍然在現場。

Fully Local ·Self-Healing ·Policy-Constrained

NemoClaw × Nemotron × TensorRT-LLM × Local RAG
Jetson AGX Thor

黃老大帶我飛



Operational intelligence belongs
next to the infrastructure it protects.

監控者不能與被監控者共命運

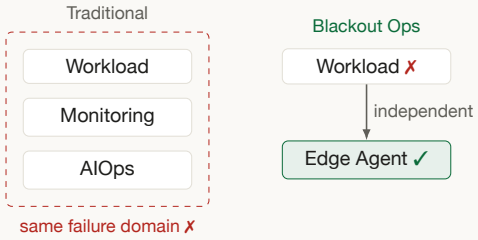
Application Value 30% ·WHY EDGE

今天的問題不是 AI 不夠聰明，而是維運系統可能跟著故障一起消失。

三個 Failure Domains

- **Cluster Down**：監控服務一起被驅逐
- **Network Degraded**：遠端管理與雲端 AI 失效
- **Air-Gapped**：外部服務從一開始就不准連

BMC 把控制面移出 Host。
Blackout Ops 把推理能力也移出 Host。



對照徵件：四項能力全中

Readability 10% ·評審清單直接回答

每一項能力都有量測過的機制，不是提示詞。

徵件要求	我們的機制	頁
目標與任務分解（必備）	證據驅動的事故迴圈，9/9 自主修復	P4
工具與系統調用	20 個白名單工具，沙箱內經 MCP 直接調用	P5
狀態與動態調整	動作後健康推送、前置條件改路線、四層調整	P6
權限與風險控制	R/W/D 分級·雙鑰匙審核·fail-closed	P7

Edge Optimization 35%	**Agent Architecture 25%**	**Application Value 30%**	**決賽要件**
Benchmark 比對 P9 · 量測計畫 P10	P4—P8	場景 P2 ·現場驗證 P11	NemoClaw ·Nemotron ·TensorRT-LLM ·Thor

下一步不是預先寫好的

能力一·目標與任務分解（必備）

已量·DGX Spark

每一步都由上一步的證據決定。




```
docker_ps      → Exited(1)    ⇐ 附上 runbook
docker_logs    → [emerg] …"return" (x6)
write_config   → written
docker_restart → front 200 ·api 200  停
```

真實演練 transcript，bad-config 案例

9 / 9

9 種注入故障·自主修復

48 s

中位數

- 證據工具把 runbook 一起推進上下文
- 沒看過 runbook，動作一律被拒
- 修不好換路徑，修不了就誠實升級

少一點工具，更好的 Agent

能力二·工具與系統調用

已量·DGX Spark

我們原本以為模型不夠強，後來發現是 Action Space 太亂。

BEFORE

- 24 tools 藏在 3 個 meta-tools 後面
- 16 個無關 skills
- 28,859 prompt chars
- 通過要靠後備，2.5–3 分鐘

AFTER

- 20 direct tools，經 MCP 直接調用
- 1 個 ops skill
- 14,200 prompt chars
- 9/9 自主，中位 48 s

−51%
Prompt footprint

0
猜錯 tool id

20
白名單工具

Edge Agent 優化不只是在壓模型，也要壓縮它必須思考的選項。

被擋下就換路，修好了就停

能力三·狀態與動態調整

已量·DGX Spark　規劃·決賽·Thor

Failure changes the plan. The plan is not the failure script.

四層動態調整

計畫層	修復失敗 → 換假設路徑	已量·DGX Spark
模型層	信心不足 → 升 Tier	規劃·決賽·Thor
硬體層	偵測異常 → nvpmodel 拉升	規劃·決賽·Thor
通道層	網路降級 → 審批改本地 UI	規劃·決賽·Thor

- 每個動作的結果自帶健康檢查：修好就停，沒修好就說哪個容器還壞
- 前置條件即時判斷：上游不在網路上就拒絕重啟前端，改走 network_repair

201 s
同一案例：靠模型自覺

54 s
驗證推進動作結果後

Self-Healing ≠ Root Shell

能力四·權限與風險控制

已量·DGX Spark

Agent 可以修系統，但不能碰整個世界。

ALLOWED

- 重建遺失的容器
- 受控的網路修復
- 專案自己的 log 清理
- 殺掉範圍內的失控行程

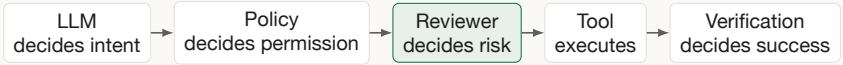
BLOCKED

- 重啟 Docker daemon
- 任意系統服務
- 共用 image 刪除
- 共用 volume 刪除

雙鑰匙審核

- 鑰匙一（程式）：充分條件缺一即拒
- 鑰匙二（第二代理）：只判相關與反證
- 連不上就不執行

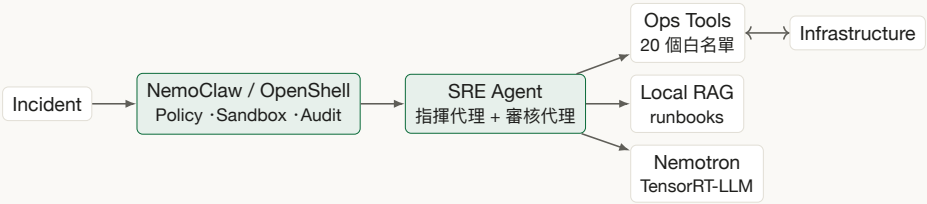
實測：無證據殺行程 **REJECT** ·量到 100% CPU **APPROVE** ·重啟仍 500 的重建 **3/3 APPROVE**



所有核心能力，都在 Edge Node 裡

Agent Architecture 25% ·NeMo 深度整合

已量·DGX Spark



Model reasons. RAG remembers. Tools observe reality. Policy controls impact.

NemoClaw 是執行基材：沙箱、政策、審計、本地推論。

決策層全是自建程式碼：迴圈、閘門、前置條件、審核、計分卡。

模型不是「挑」的，是量出來的

Edge Optimization 35% ·Benchmark 比對

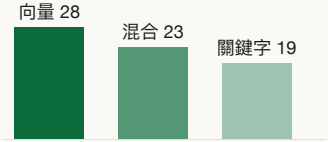
已量·DGX Spark

引擎對照：同一台、同一套腳本

	vLLM 探索組	**TensorRT-LLM 正式**
啟動到 health	275 s	**106 s**
12.9k prefill	3.5 s	**1.17 s**
GSM8K	92.7%	**96.7%**
單流 tok/s	60.7	57.9
GPU 記憶體	52 GB	**34 GB**

探索組 Qwen3.6-35B-A3B FP8 ＋ MTP；正式 Nemotron-3-Nano-30B-A3B NVFP4。7 組選型組合全數實測後定案。

檢索：向量勝出



我們原以為混合會贏。
實測後：**Vector won.**

也敢關掉沒用的優化：PDL、NGram、block reuse。

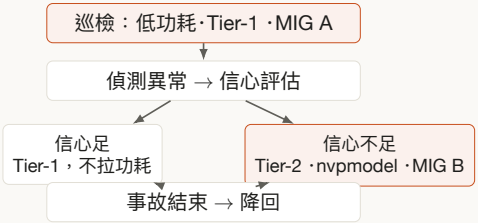
Edge inference is a memory-movement problem, not just a FLOPS problem.

算力不是限制條件，是 Agent 自己要管理的資源

Edge Optimization 35% ·量測計畫

規劃·決賽·Thor　已量·DGX Spark

全場每隊都拿到 128 GB；帶外裝置的真實成本是累積能耗。



量什麼、跟誰比

指標	對照組
單次診斷能耗 Wh	同機、全程高功耗
24 h 累積能耗 Wh	1U 監控伺服器
每瓦診斷次數	x86 工作站＋獨顯
壓縮比 vs 正確率	無壓縮

Tier-1 = 關 thinking，Tier-2 = 開 thinking。已量：每次事故 1.5k–10k token，幾乎全在 thinking——第一刀切在這裡。

我們最後評估的不是回答

Application Value 30% ·現場驗證

已量·DGX Spark　規劃·決賽·Thor

而是：Did the system recover?

9/9
自主修復

0
弄更糟·假宣稱

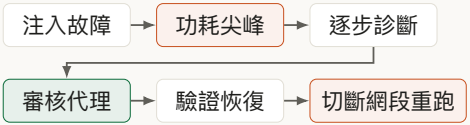
86
場演練

計分卡看五種結果

自主·靠後備·誠實升級·弄更糟·沉默失敗

單一通過率看不見前三種。另量效率、證據先行、收尾、token。

現場 demo，不預錄



每個失敗都從模型的 thinking 追根因，修的都是工具。
改進是量出來的，不是寫出來的。

三項貢獻

1
機制取代提示詞
證據先行、runbook 推送、前置條件全在工具層強制，小模型也能穩定自主修復。

2
雙鑰匙審核
程式判充分條件、第二代理判相關與反證；破壞性動作 fail-closed。

3
算力是 Agent 管理的資源
信心分級 Tier 路由與 nvpmodel/MIG 連動，能耗成為決策對象（決賽交付）。

既有成果揭露：2026/09/21–09/29 於實驗室 DGX Spark 建置並量測——NemoClaw 沙箱＋TensorRT-LLM、20 個工具、11 篇 runbook、9 種故障演練、審核代理、計分卡。 **決賽新增**：Jetson Thor 部署、能耗治理與量測、Tier 路由、壓縮曲線、本地審批 UI。

OBSERVE LOCALLY.　REASON LOCALLY.　RECOVER LOCALLY.